

プログラミング入門2

第5回

関数

篠埜 功

今日の内容 --- 関数

- 関数(function)とは、プログラムの処理の一部に名前をつけるための機構である。

例 1

```
#include<stdio.h>
int main (void) {
    int x, y;
    x = (10 + 20) * 2;
    y = (15 + 25) * 2;
    printf ("x=%d, y=%d\n",x, y);
    return 0;
}
```

```
#include <stdio.h>
int addTwice (int x, int y) {
    return (x + y) * 2;
}
int main (void) {
    int x, y;
    x = addTwice (10, 20);
    y = addTwice (15, 25);
    printf ("x=%d, y=%d\n",x, y);
    return 0;
}
```

赤字の部分の2つの式は、2つの数を足して2倍するという部分が共通している。これにaddTwiceという名前をつけてそれを使うようにしたのが右のプログラムである。

関数定義の構文

関数は、基本型(これまでに紹介したものではint, double)を引数とし、基本型を返り値とする場合、以下の形で定義される。

返り値の型名 関数名 (仮引数宣言列) 複合文

複合文の部分が、関数の本体である。

仮引数宣言列は、仮引数宣言1つか、あるいは仮引数宣言がコンマで区切られて並んだものである。

仮引数宣言は、以下の形のものである。

型名 変数名

返り値がない場合は返り値の型名の部分にvoidと書く。

引数がない場合は仮引数宣言列の部分にvoidと書く。

return文の構文1

return文の構文1

```
return 式 ;
```

return文 `return e;` の意味

`return e;` が実行されると、関数を呼び出した箇所（関数呼び出し式という）に戻る。その際、式`e`の評価結果が関数呼び出し式の値となる。

関数の本体の中でreturn文は複数個所にあってもよい。また、返り値の型名がvoid（つまり返り値無し）の関数の本体中に現れてはならない。

例2

```
#include<stdio.h>

int add (int x, int y, int z) {
    return x + y + z;
}

int main (void) {
    int a;
    a = add (2,3,4);
    printf ("a=%d¥n", a);
    return 0;
}
```

赤字の部分はreturn文であり、このreturn文が実行されると関数の呼び出し元へ戻る。その時に、 $x+y+z$ の値が返される。

return文の構文2

return文の構文2

```
return ;
```

return文 `return;` の意味

`return;` が実行されると、その関数を呼び出した箇所(関数呼び出し式という)に戻る。

返り値なしのreturn文は、返り値の型名がvoid(つまり返り値無し)の関数の中でのみ用いることができる。

返り値なしの関数においてはreturn文はなくてもよい。

return文がない場合は、関数本体の複合文が終了したときに終了する。

例3

```
#include <stdio.h>

void hello (void) {
    printf ("hello¥n");
    return;
}

int main (void) {
    hello ();
    return 0;
}
```

helloは、引数無し、返り値なしの関数であり、helloと表示するだけの関数である。

赤字の部分の関数呼び出し式hello () は値を持たないので、代入式の右辺などに書いてはならない。書いた場合の動作は未定義(undefined)。

関数呼び出し式

関数呼び出し式の構文

関数名 (引数列)

引数列は、式が0個以上並んだものである。2個以上の場合はコンマで区切る。

関数呼び出し式 $f(e_1, e_2, \dots, e_n)$ の意味

$e_1, e_2, \dots, e_n$ を評価し、それぞれの評価結果を関数 f の仮引数に代入し、関数 f の本体の複合文を実行する。返り値のある関数の場合、返り値が関数呼び出し式の値となる。

例4

```
#include<stdio.h>
int addTwo (int x) {
    return x+2;
}
int main (void) {
    int a;
    a = addTwo (2+1);
    printf ("a=%d¥n", a);
    return 0;
}
```

この例の場合、関数呼び出し式 `addTwo (2+1)` の評価は、まず引数の `2+1` を評価し、その結果である `3` を関数 `addTwo` の仮引数 `x` に代入し、関数本体の複合文を実行するという順で行われる。関数本体の中で `return` 文があり、`x+2` の評価結果の `5` が関数の返り値として返される。その値 `5` が、関数呼び出し式 `addTwo(2+1)` の値となり、それが `a` に代入され、`a=5` が表示される。

仮引数の有効範囲

```
#include<stdio.h>
int addTwo (int x) {
    return x+2;
}
int main (void) {
    int a;
    a = addTwo (2+1);
    printf ("a=%d¥n", a);
    return 0;
}
```

関数addTwoの仮引数xの有効範囲は、addTwoの本体の複合文(赤字部分)である。
(ただしaddTwoの本体の複合文の中にさらに複合文を書いてその中でxを宣言した場合は、そのxの有効範囲は除く。)

関数の例5 (打ち込んで確認)

```
#include<stdio.h>
int addTwo (int x) {
    x = x + 2;
    return x;
}
int main (void) {
    int a, x = 3;
    a = addTwo (x);
    printf ("x=%d\n", x);
    printf ("a=%d\n", a);
    return 0;
}
```

この例の場合、関数呼び出し式 `addTwo (x)` の評価は、まず引数の `x` (赤色) を評価し、その結果である3を関数 `addTwo` の仮引数 `x` (緑色) に代入し、関数本体の複合文を実行するという順で行われる。関数本体の中の代入式 `x=x+2` の部分で `x` (緑色) に5が代入され、`return` 文 `return x;` で `x` (緑色) の評価結果の5が関数の返回值として返される。その値5が、関数呼び出し式 `addTwo(x)` の値となり、それが `a` に代入される。そして、`x=3, a=5` が表示される。

`addTwo` の引数 `x` (緑色) と `main` 関数中の `x` (赤色) は別の変数である。

例6

```
/* 1からnまでの和  
を表示 */  
#include <stdio.h>  
int sum (int n) {  
    int i=1, sum=0;  
    while (i<=n) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    return sum;  
}
```

```
/* 左の続き */  
int main (void) {  
    int n;  
    printf ("自然数を入力してください: ");  
    scanf ("%d", &n);  
    printf ("1から%dまでの和は%dです。¥n",  
           n, sum (n));  
    return 0;  
}
```

関数定義の本体は複合文であり、これまでmain関数の本体で書いていたように変数を宣言したりwhile文を使ったりして、自由にプログラムを書くことができる。

例7

```
/* 1からnまでの和を表  
示 */  
#include <stdio.h>  
int sum (int n) {  
    if (n<=0)  
        return 0;  
    else  
        return n + sum (n-1);  
}
```

```
/* 左の続き */  
int main (void) {  
    int n;  
    printf ("自然数を入力してください: ");  
    scanf ("%d", &n);  
    printf ("1から%dまでの和は%dです。¥n",  
           n, sum (n));  
    return 0;  
}
```

関数sumの中で、関数sumを呼び出している。このような関数を再帰関数という。関数呼び出しには仮引数への代入等に時間が少しかかるので、例6のようにループで回した方が一般に実行速度は速い。

引数の評価順序について

関数呼び出し式 $f(e_1, e_2, \dots, e_n)$ の評価においては式 $e_1, e_2, \dots, e_n$ の評価がまず行われるが、これらの式の評価順序は未規定 (unspecified) である。

関数の引数の式には副作用のある式は書かないようにする。

プロトタイプ宣言

- 関数定義が、関数呼び出し場所より後にある場合はプロトタイプ宣言を関数呼び出しより前で行わなければならない。

```
#include<stdio.h>
int main (void) {
    double x, y;
    x = addTwice (10.0, 20.5);
    y = addTwice (15.0, 25.0);
    printf ("x+y=%f¥n", x+y);
    return 0;
}
double addTwice (double x, double y) {
    return (x + y) * 2;
}
```

この例ではaddTwiceの関数定義が使用箇所より後ろにある。このような場合、関数の返り値はint型として処理される(ので型が合わなくなる)。プロトタイプ宣言が必要。

例8 (打ち込んで確認)

```
#include<stdio.h>
double addTwice (double, double);

int main (void) {
    double x, y;
    x = addTwice (10.0, 20.5);
    y = addTwice (15.0, 25.0);
    printf ("x+y=%f¥n", x+y);
    return 0;
}
double addTwice (double x, double y) {
    return (x + y) * 2;
}
```

赤字の部分がプロトタイプ宣言。関数の返り値の型および引数の型を記述する。

C言語プログラムの実行について

- C言語プログラムは、main関数から実行が開始される。

main関数について

- main関数の返り値はint型
- main関数の引数はない場合とある場合がある。この授業では引数無しのmain関数のみを扱う。

main関数はこの演習では以下の形で定義する。

```
int main (void) { ... }
```

main関数の返り値について

- (最初に呼ばれる)main関数の返り値はプロセスの終了statusとなる。(main関数の呼び出し元(スタートアップルーチン)でexit関数の引数に渡される。)
 - [補足] main関数は他の関数から呼び出したり、main関数内から呼び出したりすることもできる。
- C99からは、main関数の実行が関数本体のブロックの終了まで行き着いた場合、0を返り値として返す。C89では、この場合の終了statusは未定義。

main関数はこの演習ではreturn 0;で終了させるようにする。

補足

- 関数は、他の言語では手続き(procedure)と言うこともある。
- 値を返さない場合は手続き、値を返す場合は関数と呼ぶのが普通だと思われるが、区別しないで使う場合もある。Cでは値を返さない関数も定義できる(返り値の型の部分をvoidと記述すればよい)。

基本課題1

円の半径をキーボードからdouble型で読み取り、円周をdouble型で以下の実行例のように表示するプログラムを作成せよ。ただし、半径をdouble型の引数で受け取り、円周を結果として返す関数circumferenceを以下のように定義し、それを用いたプログラムとせよ。

```
double circumference (double radius) { ... }
```

円周は、 $2 \times \text{半径} \times \text{円周率}$ で求められる。円周率は3.14とする。

[実行例]

円の半径を入力: 3.5

円周は21.98です。

[補足] scanfの変換指定子は%lf、printfの変換指定子は%.2fとしてください。

基本課題2

身長(cm)と体重(kg)をキーボードからdouble型で読み取り、BMIをdouble型で以下の実行例のように表示するプログラムを作成せよ。ただし、身長(cm)と体重(kg)をdouble型の引数で受け取り、BMIを結果として返す関数bmiを以下のように定義し、それを用いたプログラムとせよ。

```
double bmi (double height, double weight) { ... }
```

BMIの計算式は、 $\text{体重(kg)} / (\text{身長(m)} \times \text{身長(m)})$ である。身長の単位がcmではなく、mであることに注意する。

[実行例]

身長(cm): 176.0

体重(kg): 70.0

BMIは22.6です。

[補足] scanfの変換指定子は%lf、printfの変換指定子は%.1fとしてください。

発展課題1

0以上の整数 n をキーボードからint型で読み取り、フィボナッチ数列の n 番目の数 $F(n)$ を以下のように表示するプログラムを作成せよ。ただし、int型の数を引数として受け取り、 $F(n)$ を返す関数fibonacciを以下の形で定義し、それを用いたプログラムとせよ。

```
int fibonacci (int n) { ... }
```

フィボナッチ数は以下のように定義される。

$$F(0)=1, F(1)=1, F(n)=F(n-1)+F(n-2) \quad (n \geq 2)$$

[実行例]

0以上の整数を入力: 10

$F(10)=89$

[注意事項] fibonacci関数を上記の定義に従って再帰的に定義すると計算時間が引数に与えられる数に関して指数オーダーになり、 $F(100)$ などの計算は非常に長い時間かかるが、この演習ではその回答でもOKとする。

発展課題2

キーボードから正の整数を2つ読み取り、それらの最大公約数を表示するプログラムを作成せよ。ただし、 n, m を引数にとり、 n, m の最大公約数を結果として返す関数gcdを以下のように定義し、それを用いたプログラムとせよ。

```
int gcd (int n, int m) { ... }
```

[実行例]

正の整数を入力してください: 12

正の整数を入力してください: 18

12と18の最大公約数は6です。

発展課題3

キーボードから正の整数を2つ読み取り、それらの最小公倍数を表示するプログラムを作成せよ。ただし、 n, m を引数にとり、 n, m の最小公倍数を結果として返す関数 `lcm` を以下のように定義し、それを用いたプログラムとせよ。

```
int lcm (int n, int m) { ... }
```

[実行例]

正の整数を入力してください: 12

正の整数を入力してください: 18

12と18の最小公倍数は36です。

[ヒント]

発展課題2の関数 `gcd` を使って `lcm` を定義すると分かりやすい。

発展課題4

キーボードから正の整数を読み取り、その数が素数かどうかを表示するプログラムを作成せよ。ただし、 n を引数にとり、 n が素数の場合1, そうでない場合0を返す関数isPrimeを以下ように定義し、それを用いたプログラムとせよ。

```
int isPrime (int n) { ... }
```

[実行例1]

正の整数を入力してください: 10

10は素数ではありません。

[実行例2]

正の整数を入力してください: 11

11は素数です。

[補足] 素数とは、1と自分自身以外に正の約数を持たない、1でない正の整数のことである。

参考課題1

三角形の底辺と高さをキーボードからdouble型で読み取り、面積(底辺 × 高さ/2)をdouble型で表示するプログラムを作成せよ。ただし、底辺と高さをdouble型の引数2つで受け取り、面積を結果として返す関数triangleAreaを以下のように定義し、それを用いたプログラムとせよ。

```
double triangleArea (double base, double height) { ... }
```

[実行例]

三角形の底辺を入力: 3.5

三角形の高さを入力: 4.5

三角形の面積は7.9です

参考課題1 解答例

```
#include <stdio.h>
double triangleArea (double base, double height) {
    return base * height / 2;
}
int main (void) {
    double base, height;
    printf("三角形の底辺を入力: ");
    scanf("%lf", &base);
    printf("三角形の高さを入力: ");
    scanf("%lf", &height);
    printf("三角形の面積は%.1fです¥n", triangleArea(base,height));
    return 0;
}
```

参考課題2

キーボードから正の整数を読み取り、1からその数までの2乗の和($1^2 + 2^2 + 3^2 \dots$)を表示するプログラムを作成せよ。ただし、nを引数にとり、1からnまでの2乗和を結果として返す関数squareSumを以下のように定義し、それを用いたプログラムとせよ。

```
int squareSum (int n) { ... }
```

[実行例]

正の整数を入力してください: 10

1から10までの2乗和は385です。

参考課題2 解答例

```
#include <stdio.h>
int squareSum (int n) {
    int i,r=0;
    for (i=1; i<=n; i=i+1)
        r=r+i*i;
    return r;
}
int main (void) {
    int n;
    printf ("正の整数を入力してください: ");
    scanf ("%d", &n);
    printf ("1から%dまでの2乗和は%dです。¥n", n, squareSum(n));
    return 0;
}
```

参考課題3

3つの整数をキーボードから読み取り、その中で最も小さい整数を表示するプログラムを作成せよ。ただし、3つの整数を引数にとり、その中の最小値を結果として返す関数min3を以下のように定義し、それを用いたプログラムとせよ。

```
int min3 (int n1, int n2, int n3) { ... }
```

[実行例]

整数を入力してください: 5

整数を入力してください: 10

整数を入力してください: 7

最小値は5です。

参考課題3 解答例

```
#include <stdio.h>
int min3 (int n1, int n2, int n3) {
    int min;
    min = n1;
    if (n2 < min)
        min = n2;
    if (n3 < min)
        min = n3;
    return min;
}
```

```
/* 続き */
int main (void) {
    int i, a[3];
    for (i=0; i<3; i=i+1) {
        printf ("整数を入力してください: ");
        scanf ("%d", &a[i]);
    }
    printf ("最小値は%dです。¥n",
           min3 (a[0],a[1],a[2]));
    return 0;
}
```